Lab Exercise 5

Task: Aligning Speech Signals Using Linear Time Normalization

In this notebook, we will align two speech signals derived from utterances of the same word using Linear Time Normalization (LTN). Signal 1 is a sampled version of "hello" spoken at a normal pace, and Signal 2 is a sampled version of "hello" spoken more slowly. Our goal is to align Signal 2 with Signal 1.

(a) Plot both speech signals to observe their differences in length and amplitude patterns

Importing Dependencies:

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d
```

Define the signals:

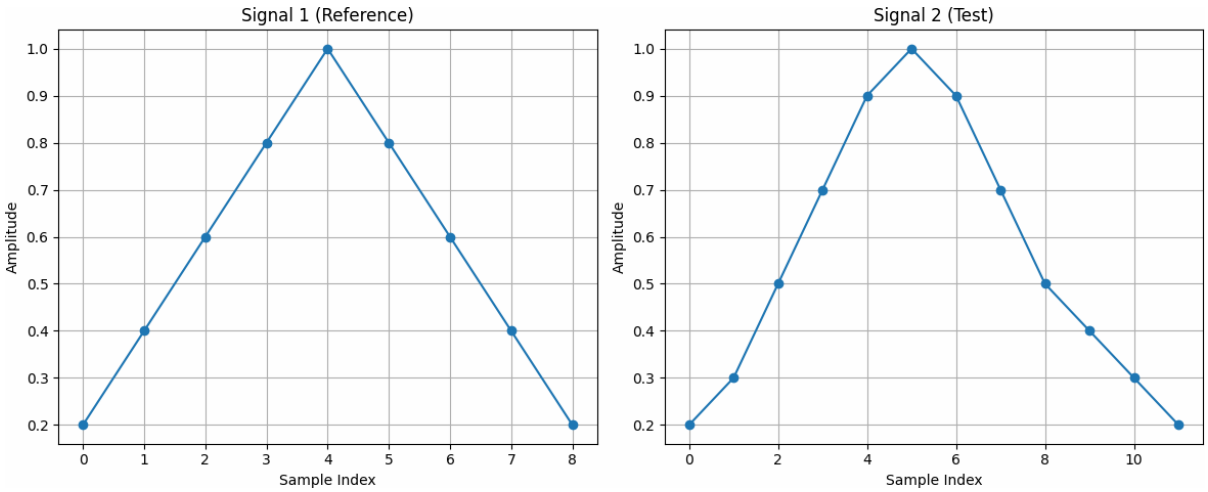
In [2]:
```
signal1 = np.array([0.2, 0.4, 0.6, 0.8, 1.0, 0.8, 0.6, 0.4, 0.2])
signal2 = np.array([0.2, 0.3, 0.5, 0.7, 0.9, 1.0, 0.9, 0.7, 0.5, 0.4, 0.3, 0
```

Plotting both the signals to observe the differences:

In [3]:
```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(signal1, marker='o')
plt.title('Signal 1 (Reference)')
plt.xlabel('Sample Index')
plt.ylabel('Amplitude')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(signal2, marker='o')
plt.title('Signal 2 (Test)')
plt.xlabel('Sample Index')
plt.ylabel('Amplitude')
plt.grid(True)

plt.tight_layout()
plt.show()
```



Observation:

From the plots, we can see that Signal 2 is longer than Signal 1, indicating that it was spoken more slowly. The amplitude patterns are similar but stretched in time.

(b) Perform Linear Time Normalization for the two sequences

To align the two signals, we will perform Linear Time Normalization by interpolating Signal 2 to match the length of Signal 1.

In [4]:
```
len1 = len(signal1)
len2 = len(signal2)

x_old = np.linspace(0, 1, len2)
f = interp1d(x_old, signal2, kind='linear') # Interpolate Signal 2 to match

x_new = np.linspace(0, 1, len1)
signal2_normalized = f(x_new)
```

Explanation:

We use linear interpolation to resample Signal 2 so that it has the same number of samples as Signal 1. This effectively compresses Signal 2 in time to match the duration of Signal 1.

(c) Compute the alignment between Signal 1 and the normalized Signal 2

Now that both signals have the same length, we can compute the alignment between them.

In [5]:
```
alignment = signal1 - signal2_normalized

mse = np.mean(alignment ** 2)
print(f"Mean Squared Error after Linear Time Normalization: {mse:.4f}")
```

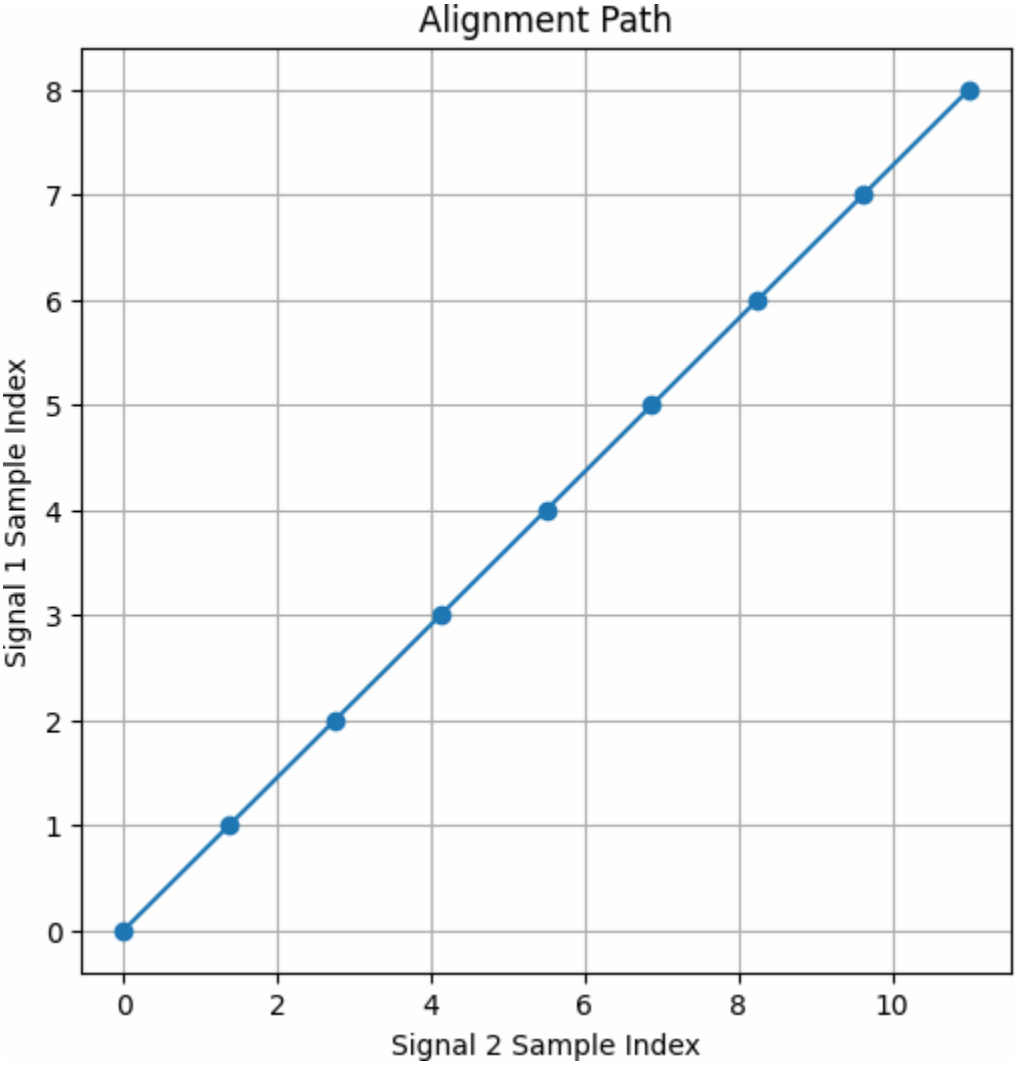
```
Mean Squared Error after Linear Time Normalization: 0.0048
```

(d) Plot the alignment path, showing how each sample in Signal 1 corresponds to a sample in Signal 2

We can visualize how samples in Signal 1 correspond to samples in the original (unaligned) Signal 2.

In [6]:
```
signal1_indices = np.arange(len1)
signal2_indices = np.linspace(0, len2 - 1, len1)

plt.figure(figsize=(6, 6))
plt.plot(signal2_indices, signal1_indices, marker='o')
plt.title('Alignment Path')
plt.xlabel('Signal 2 Sample Index')
plt.ylabel('Signal 1 Sample Index')
plt.grid(True)
plt.show()
```



Explanation:

The alignment path is a straight line, indicating a linear mapping between the sample indices of the two signals.

(e) Discuss how Linear Time Normalization aligns the two speech signals (inference)

Linear Time Normalization aligns the two speech signals by uniformly scaling the time axis of Signal 2 so that it matches the length of Signal 1. This method assumes that the speed difference between the two signals is consistent throughout the entire duration. By interpolating Signal 2 to have the same number of samples as Signal 1, we effectively compress its time scale.

However, LTN may not account for non-linear variations in speaking speed within different parts of the word. For example, if certain phonemes are elongated more than others, LTN may not perfectly align the signals. In such cases, more sophisticated methods like Dynamic Time Warping (DTW) may provide better

alignment by allowing for non-linear mappings between the time axes of the signals.

Task: Analyzing Alignment of Two Vectors Using Dynamic Time Warping (DTW)

In this notebook, we will use Dynamic Time Warping (DTW) to compare and align two numerical sequences. DTW is a powerful algorithm for measuring similarity between two sequences that may vary in time or speed. It finds the optimal alignment between the sequences by warping the time dimension.

Given Vectors:

- Vector 1: [2, 3, 4, 6, 8, 7, 6, 5, 4, 3, 2]
- Vector 2: [2, 4, 6, 7, 7, 6, 5, 5, 4, 3, 2, 2, 1]

Vector 2 is a stretched and slightly shifted version of Vector 1. Our goal is to analyze the alignment of these two vectors using DTW.

(a) Plot both vectors to visualize their patterns.

Importing Dependencies

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

Define the vectors:

In [2]:
```
vector1 = np.array([2, 3, 4, 6, 8, 7, 6, 5, 4, 3, 2])

vector2 = np.array([2, 4, 6, 7, 7, 6, 5, 5, 4, 3, 2, 2, 1])
```

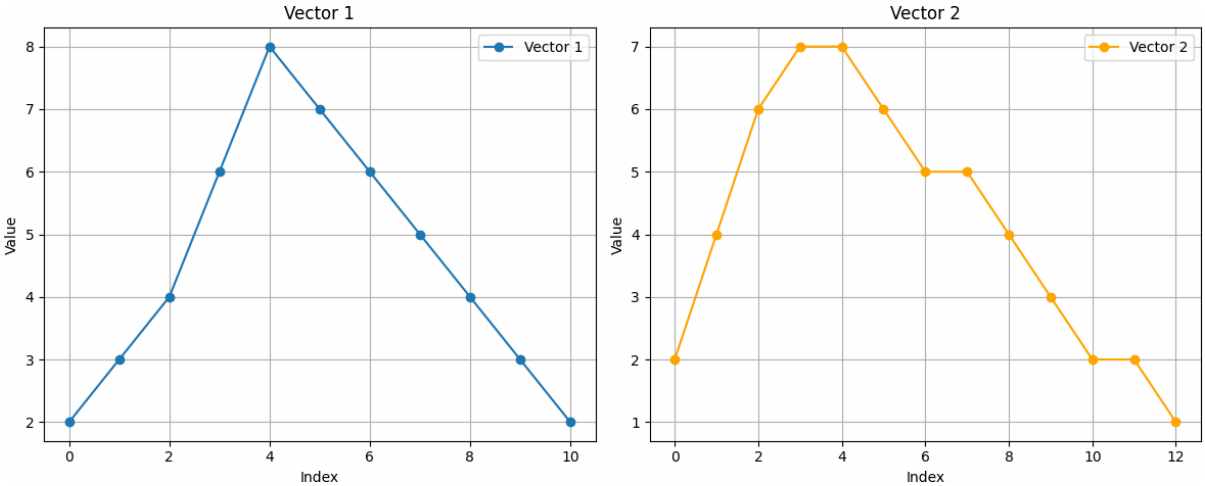
Plotting both the signals to visualize their patterns:

In [3]:
```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(vector1, marker='o', label='Vector 1')
plt.title('Vector 1')
plt.xlabel('Index')
plt.ylabel('Value')
plt.grid(True)
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(vector2, marker='o', color='orange', label='Vector 2')
plt.title('Vector 2')
```

```
plt.ylabel('Value')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```



Observation:

From the plots, we can see that Vector 2 is longer than Vector 1 and appears to be a stretched and slightly shifted version of Vector 1. The overall patterns are similar but vary in time.

(b) Implement Dynamic Time Warping (DTW) algorithm

Defining the DTW Function:

In [4]:
```
def dtw(vector1, vector2):
    n = len(vector1)
    m = len(vector2)
    dtw_matrix = np.full((n+1, m+1), np.inf)
    dtw_matrix[0, 0] = 0
    for i in range(1, n+1):
        for j in range(1, m+1):
            cost = abs(vector1[i-1] - vector2[j-1])
            dtw_matrix[i, j] = cost + min(
                dtw_matrix[i-1, j],
                dtw_matrix[i, j-1],
                dtw_matrix[i-1, j-1]
            )
    return dtw_matrix
```

Explanation:

- Initialization: Create a (n+1) x (m+1) matrix filled with infinity, and set dtw_matrix[0, 0] = 0.

- Cost Calculation: For each cell (i, j), compute the cost as the absolute difference between vector1[i-1] and vector2[j-1].
- Update DTW Matrix: Update the dtw_matrix[i, j] with the cost plus the minimum of the neighboring cells (insertion, deletion, match).

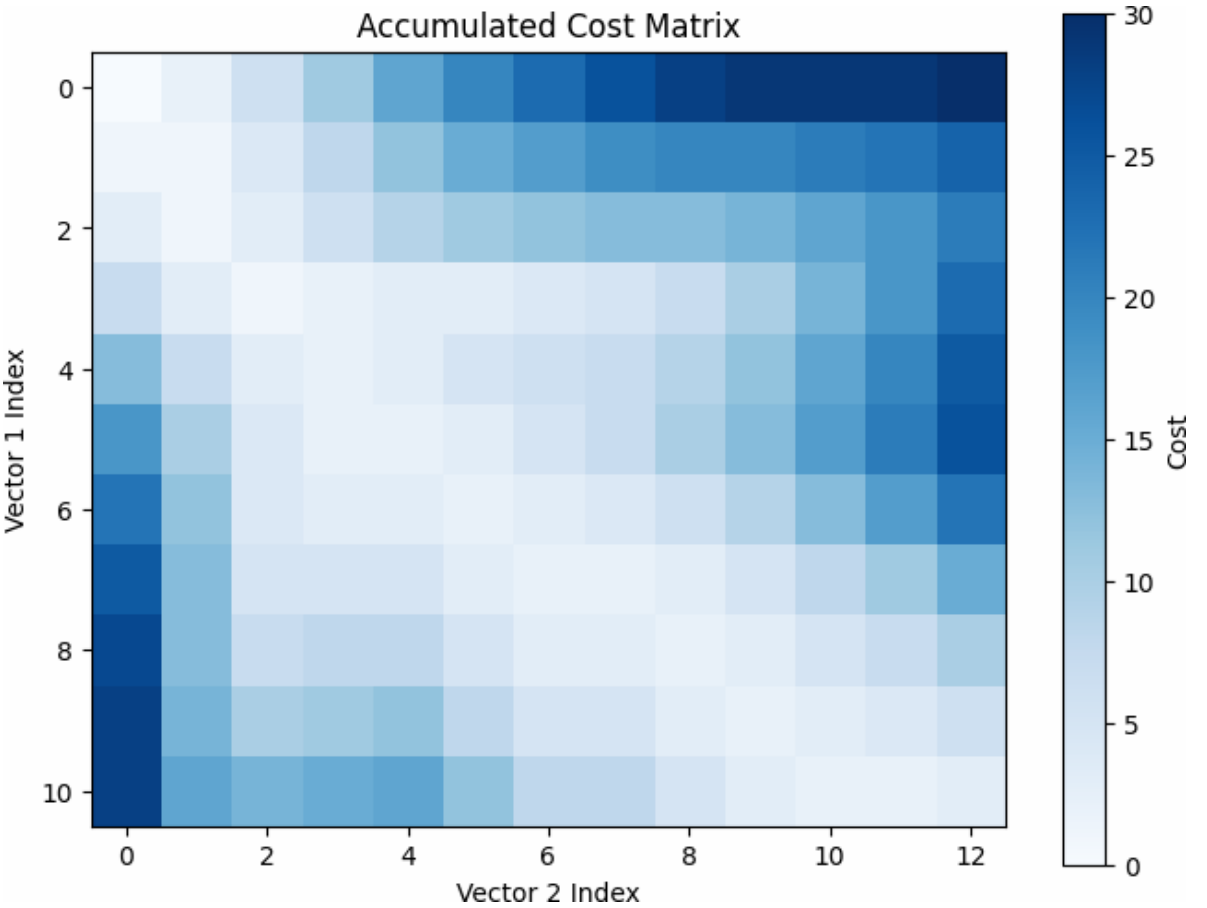
(c) Compute the accumulated cost matrix.

Compute the accumulated cost matrix using the DTW algorithm:

In [5]:
```
dtw_matrix = dtw(vector1, vector2)
```

Visualizing the accumulated cost matrix:

In [6]:
```
plt.figure(figsize=(8, 6))
plt.imshow(dtw_matrix[1:,1:], interpolation='nearest', cmap='Blues')
plt.title('Accumulated Cost Matrix')
plt.xlabel('Vector 2 Index')
plt.ylabel('Vector 1 Index')
plt.colorbar(label='Cost')
plt.show()
```



Explanation:

The accumulated cost matrix shows the cumulative cost of aligning the elements of the two vectors up to each point. The optimal path will minimize this cost.

(d) Find and visualize the warping path.

To find the optimal warping path, we backtrack from the bottom-right corner of the matrix.

In [7]:
```
def dtw_path(dtw_matrix):
    i, j = np.array(dtw_matrix.shape) - 1
    path = [(i-1, j-1)]
    while (i > 1) or (j > 1):
        directions = [(i-1, j-1), (i-1, j), (i, j-1)]
        costs = [dtw_matrix[d] for d in directions]
        min_cost_index = np.argmin(costs)
        i, j = directions[min_cost_index]
        path.append((i-1, j-1))
    path.reverse()
    return path
```

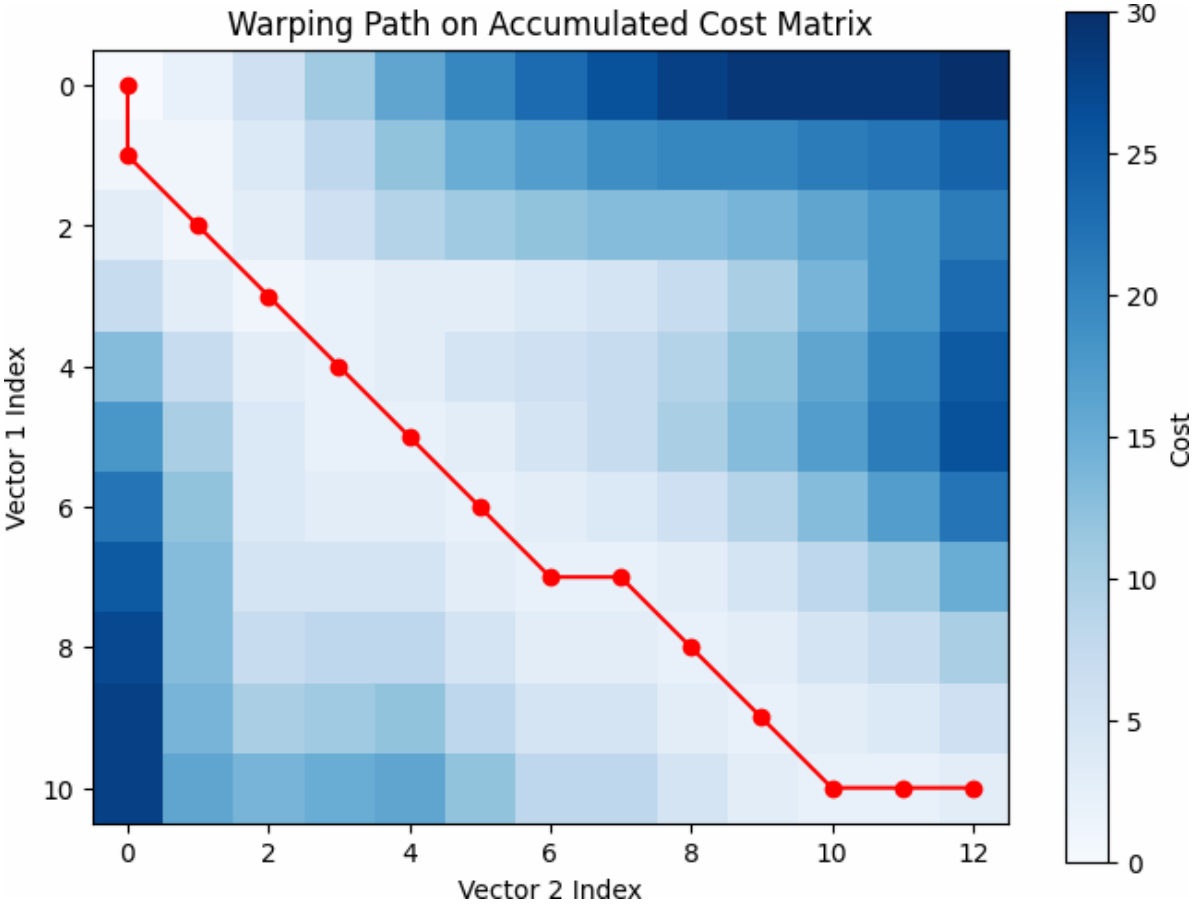
Compute the warping path:

In [8]:
```
path = dtw_path(dtw_matrix)
```

Visualize the warping path on the accumulated cost matrix:

In [9]:
```
path_x, path_y = zip(*path)

plt.figure(figsize=(8, 6))
plt.imshow(dtw_matrix[1:,1:], interpolation='nearest', cmap='Blues')
plt.plot(path_y, path_x, '-o', color='red')
plt.title('Warping Path on Accumulated Cost Matrix')
plt.xlabel('Vector 2 Index')
plt.ylabel('Vector 1 Index')
plt.colorbar(label='Cost')
plt.show()
```



Explanation:

- Backtracking: Starting from the bottom-right corner, we move to the neighboring cell with the minimum accumulated cost.
- Warping Path: The path represents the optimal alignment between the vectors.
- Visualization: The red line on the accumulated cost matrix shows the warping path.

(e) Calculate the DTW distance between the vectors.

The DTW distance is the value in the bottom-right cell of the accumulated cost matrix.

In [10]:
```
dtw_distance = dtw_matrix[-1, -1]
print(f"DTW Distance between Vector 1 and Vector 2: {dtw_distance:.2f}")
```

```
DTW Distance between Vector 1 and Vector 2: 3.00
```

Explanation:

The DTW distance quantifies the minimal total cost required to align the two vectors.

(f) Write an inference on how the warping path aligns the two vectors and what does the DTW distance indicate about their similarity?

Inference:

The warping path obtained from the DTW algorithm shows how each element in Vector 1 is aligned with elements in Vector 2. The path is not strictly diagonal, indicating that some elements in one vector are aligned with multiple elements in the other vector. This reflects the stretching and shifting between the two vectors.

- Alignment Details:
  - Stretched Segments: Sections where the path moves horizontally or vertically indicate that an element in one vector is aligned with multiple elements in the other vector.
  - Shifted Segments: Diagonal shifts in the path capture the slight shifts between the vectors.

The DTW distance of 3.00 indicates the cumulative minimal cost required to align the two vectors. A lower DTW distance suggests higher similarity, whereas a higher distance indicates greater dissimilarity.

- Similarity Interpretation:
  - Given the low DTW distance, we can infer that the vectors are quite similar in shape and pattern, despite the stretching and shifting.
  - The DTW algorithm effectively compensates for these temporal variations, allowing for a meaningful comparison.

Conclusion:

Dynamic Time Warping provides a robust method for aligning sequences that may vary in time or speed. By computing the optimal warping path and DTW distance, we can quantify and visualize the similarity between two vectors, even when they are not perfectly aligned in time.